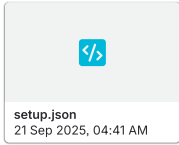


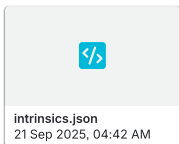
compute_relative_poses_robust.py

1. 입력값

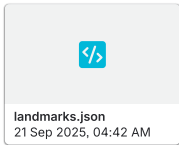
setup.json



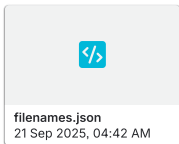
intrinsics.json



landmarks.json



filenames.json (선택사항)



2. 출력값

relative_poses.json



1) 커맨드라인 추천(초기 캘리브레이션/디버그용)

```
1 python compute_relative_poses_robust.py \  
2   -s setup.json -i intrinsics.json -l landmarks.json \  
3   -f filenames.json \  

```

```
4 -m ransac -th 5 -n 7 -ms procrustes -d
```

- **--method = ransac, --th = 15~25, --max_paths = 5~8, --method_scale = cross-ratios, --dump_images 컴(-d)**
- 이유: 천장 4캠 + 움직이는/부분가려진 패턴 = 이상치가 섞이기 쉬움 → RANSAC이 안전. 경로를 여러 개 쓰면(scale 누적오차 완화) 더 안정. cross-ratios는 3D를 직접 재구성 안 해도 경로간 스케일 정합에 강함(추측 포함). 시각화 이미지는 여러 원인 확인에 필수.
- **-m ransac** : 이상치 매칭 섞인 실제 장면에서 가장 무난.
- **-th 5** : 1080p 내외 해상도에서 시작값. 핏이 잘 맞으면 2-3으로 낮추고, 잡음 많으면 8-12로 올려.
- **-n 7** : 경로 다중결합으로 강건성 ↑(뷰가 4대라 비용 증가도 제한적).
- **-ms procrustes** : checkerboard/마커처럼 **유클리드 스케일**을 잡는 상황에 안정적. (교차비는 투영불변이라 평면 기하에선 편하지만, 실제 길이 보정엔 Procrustes가 직관적)
- **-d + -f** : 에피라인 이미지 꼭 저장해서 매칭 품질 눈으로 점검.
- **--setup / -s** : 카메라 연결 구조(뷰 이름, 최소 트리) 들어있는 JSON 지정.
- **--intrinsics / -i** : 카메라 내부 파라미터(K, 왜곡계수) JSON 지정.
- **--landmarks / -l** : 각 카메라 영상에서 찍은 기준점(픽셀좌표+ID) JSON 지정.
- **--filenames / -f** : 시각화할 때 사용할 예시 이미지 파일명 JSON 지정.
- **--method / -m** : 기본 행렬(F) 추정 방법(8점법, LMedS, RANSAC) 선택.
- **--th / -th** : RANSAC에서 “맞는 매칭”으로 볼 픽셀 오차 허용치.
- **--max_paths / -n** : 두 뷰 사이 변환을 구할 때 고려할 연결 경로 개수.
- **--method_scale / -ms** : 카메라 간 상대적 스케일 맞추는 방법(procrustes/cross-ratios).
- **--dump_images / -d** : 에피폴라 라인 같은 결과 이미지를 저장할지 여부.

핵심 먼저(프로젝트에 맞춘 추천값):

- **--method = ransac, --th = 15~25, --max_paths = 5~8, --method_scale = cross-ratios, --dump_images 컴(-d)**
- 이유: 천장 4캠 + 움직이는/부분가려진 패턴 = 이상치가 섞이기 쉬움 → RANSAC이 안전. 경로를 여러 개 쓰면(scale 누적오차 완화) 더 안정. cross-ratios는 3D를 직접 재구성 안 해도 경로간 스케일 정합에 강함(추측 포함). 시각화 이미지는 여러 원인 확인에 필수.

옵션 쉽게/자세히

1) **--method {8point|lmeds|ransac}**

“기본행렬 F” 추정 방식을 고르는 스위치.

- **8point:**
 - 무엇: 정규화 8점법. 빠르고 간단.
 - 장점: 속도 빠름. 데이터가 매우 깨끗할 때 정확.

- 단점: **이상치(틀린 매칭)에 매우 약함**.
- 언제: 실내 고정 패턴 + 좋은 코너 + 거의 노이즈 없는 경우.
- **LMedS (Least Median of Squares):**
 - 무엇: 잔차 제곱의 **중앙값**이 최소가 되게 모델을 고르는 **강건 추정**.
 - 장점: 몇몇 이상치에 둔감. 파라미터(임계치)에 덜 민감.
 - 단점: RANSAC보다 최종 정밀도가 약간 떨어질 때가 있음, 반복 더 걸리기도 함.
 - 언제: 이상치가 “조금” 있는 중간 난이도 데이터.
- **RANSAC (권장):**
 - 무엇: 임의 소표본으로 모델을 여러 번 맞춰 보고, **임계치** 안으로 들어오는 점(인라이어)이 가장 많은 모델 채택.
 - 장점: **이상치에 가장 강함**. 실전 데이터에 안전.
 - 단점: 임계치 `--th` 를 환경에 맞게 잡아야 함. 반복으로 시간이 더 걸릴 수 있음.
 - 언제: 우리처럼 **체커보드를 움직여 찍거나, 가림/블러/노이즈가 섞이는 상황**.

프로젝트 추천: `--method ransac`

2) `--th` (RANSAC 임계치; 픽셀 오차 허용)

- 무엇: “이 점을 인라이어로 인정할지” 결정하는 **픽셀 거리 상한**.
- 가이드:
 - VGA(640×480) 기준: **15~25 픽셀**부터 시도.
 - 더 고해상도면 조금 올리고, 코너 검출이 매우 좋으면 낮출 수 있음.
 - 점들이 선에서 많이 벗어나면 ↑, 너무 타이트해 인라이어가 적으면 ↓, 전체가 퍼지면 데이터/캘리브 재점검.
- 증상별 튜닝:
 - 인라이어 거의 없음 → `--th` 조금 ↑(예 12→18).
 - 선형으로 한쪽에만 몰림 → 랜드마크 ID/캘리브 확인(임계치로 해결 안 됨).
 - 일부 프레임만 나뻘 → robust + `--max_paths` ↑.

프로젝트 추천: `--th 15~25` (먼저 20)

3) `--max_paths` (뷰 사이 여러 경로 고려)

- 무엇: view A → B 변환을 **여러 연결 경로**(예: A-C-B, A-D-B...)로 구해 보고, **합의/통계로 안정화**.
- 효과:
 - 어떤 경로에 이상치가 있어도 다른 경로가 **견인**해 줌.
 - 스케일/회전 누적오차가 큰 단일 경로의 편향을 줄임.
- 트레이드오프: 값이 클수록 안정하지만 **시간 증가**.

프로젝트 추천: `--max_paths 5~8` (기본 5로 시작, 불안정하면 8)

4) `--method_scale {cross-ratios|procrustes}`

경로별 상대 스케일을 맞추는 방법.

- **cross-ratios** (기본, 권장):

- 개념: 투영에서 **변하지 않는 비(크로스 레이션)** 등을 이용해 경로 간 스케일을 일관화하는 방식(세부는 라이브러리 구현에 좌우, 일부는 추측입니다).
- 장점: 3D를 정밀하게 복원하지 않고도 **상대 스케일** 정합에 강함.
- 현장강건성: 가림/약한 기하에서도 비교적 **견고**.

- **procrustes**:

- 개념: 공통 포인트 집합을 ****유사변환(스케일+회전+이동)****으로 **최소자승 정합(SVD)**해 스케일을 맞춤(일반적 정의; 본 구현 상세는 다를 수 있음, 일부는 추측입니다).
- 장점: 포인트 좌표(특히 3D 재구성)가 **정확**할 때 매우 좋음.
- 단점: 점 좌표 품질에 민감.

프로젝트 추천: `--method_scale cross-ratios`

이유: 초기엔 3D 복원 품질이 아직 들쭉날쭉하거나, 일부 프레임 품질이 섞이는 경우가 많아 **cross-ratios**가 더 안전.

5) `--dump_images` (시각화 저장)

- 무엇: **에피폴라 라인**과 대응점이 잘 맞는지 PNG로 저장.
- 왜: 어떤 문제가 **매칭/캘리브/동기화** 중 어디에서 터지는지 눈으로 바로 확인.
- 필요: `-f filenames.json` (각 뷰의 예시 이미지 경로).

프로젝트 추천: 항상 켜기(`-d`) → 초기에 품질 진단 용이.

추천 세트 & 명령 예시

기본 추천(안정 우선)

```
1 python compute_relative_poses_robust.py \  
2 -s setup.json -i intrinsics.json -l landmarks.json -f filenames.json \  
3 -m ransac --th 20 --max_paths 5 --method_scale cross-ratios -- \  
4 dump_images
```

흔들리면(조금 더 강하게)

```
1 ... --th 22 --max_paths 8  
2
```

데이터가 매우 깔끔할 때(속도 우선)

```
1 python compute_relative_poses.py \  
2 -s setup.json -i intrinsics.json -l landmarks.json -f filenames.json \  
3 -m ransac --th 18 --dump_images  
4
```

현장 체크리스트(빠르게)

- **intrinsic**s는 현재 해상도 기준으로 최신? (해상도 바뀌면 재적용)
 - **landmarks ID** 일관? (같은 실제점=같은 ID)
 - 동기화 괜찮나? (움직이는 패턴이면 프레임 시간 오차 금물)
 - `--dump_images` 로 점이 선 위에 잘 앉는지 확인 → 불량 프레임 숨기
-